

DB and Python Framework

Total Time: 3 Hours

Theme: Social Media Profile Manager

Section A: Conceptual Understanding

1. Explain the Django Request-Response cycle and how it differs from a standard Python script execution.
2. Explain why Django Model Fields (CharField, IntegerField) are more robust for profile data than Python dynamic typing.
3. Explain how Django Forms handle automated input validation for usernames and age ranges.
4. Explain how to implement conditional logic in Django Templates to toggle account visibility.
5. Explain the difference between iterating through a Python list and a Django QuerySet.
6. Explain why the Django ORM is preferred over Python dictionaries for persistent profile storage.

Section B: Practical Tasks

1. **Define UserProfile Model:** Create a class in `models.py` inheriting from `models.Model`. Define fields such as `CharField` for the username, `IntegerField` for age, and `BooleanField` for the `is_public` status.
2. **Create ModelForm & Validation:** Implement a `UserProfileForm` extending `forms.ModelForm`. Add a `clean_age()` method to enforce custom constraints, such as ensuring the user is over 13 years old before allowing profile creation.
3. **Django Views for Persistence:** Develop a function-based or class-based view to handle `POST` requests. Use `form.is_valid()` to check inputs and `form.save()` to commit the new profile data directly to the database.

4. **Render Profiles with DTL:** Retrieve all profiles using `UserProfile.objects.all()` and pass them to a template. Use **Django Template Language (DTL)** loops (`{% for profile in profiles %}`) to display the username and age in an HTML list.

Section C: Mini Project

Social Profile Web App

Objective: Build a web-based management system using Django to handle user profiles with relational persistence and file export capabilities.

1. **Create/Edit via Django Forms** Develop a dynamic profile management system using **Django ModelForms**. This allows you to automatically generate HTML forms from your Profile model, handle data validation, and save user input directly to the database with minimal code.
2. **Relational Database Integration** Configure your app to communicate with a robust RDBMS like **PostgreSQL** or **MySQL**. Use the **Django ORM** to create a Profile table and implement a "List View" that queries all records from the database to display them on a central dashboard.
3. **"Save to File" with Context Managers** Implement an export feature using Python's `csv` module. Use a **Context Manager** (`with open(...) as file:`) to ensure proper file handling, allowing the app to fetch database records and write them into a downloadable CSV file safely.
4. **Clean URL Routing & Templates** Organize your application using a clear directory structure. Implement specific URL patterns for "List," "Create," and "Export" views, and use **Django Template Language (DTL)** to render the profile data into a clean, user-friendly interface.